

Assignment 6 – SOLUTION

Assignment 6 - Objectives:

- Manipulating 2D arrays (matrices) in x86-64 assembly code
- Practicing the function call convention (protocol) in x86-64 assembly code

Marking scheme:

- This assignment will be marked for correctness.
- A solution will be posted on Monday after the due date.

Deadline:

- Friday July 11 at 23:59:59 on CourSys and on Crowdmark.
- No extension will be granted unless for medical reasons, once the Official Medical Certificate has been completed and emailed to the instructor before Friday (due day) 5pm.

Late assignments will receive a grade of 0.

Enjoy!

Solution:

```
.globl    copy
# ***** Version 2 *****
copy:
# A in rdi, C in rsi, N in edx

# This function is not a "caller" i.e., it does not call functions
# (it is a leaf function),
# hence it does not have the responsibility of saving "caller-
# saved" registers
# such as rax, rdi, rsi, rdx, rcx, r8 and r9.
# This signifies that it can use these registers without first
# saving their content.

# Set up registers
    xorl %eax, %eax                # set eax to 0
    xorl %ecx, %ecx                # i = 0 (row index i is in ecx)
```

```

# For each row
rowLoop:
    xorl %r8d, %r8d          # j = 0 (column index j in r8d)
    cmpl %edx, %ecx          # while i < N (i - N < 0)
    jge doneWithRows

# For each cell of this row
colLoop:
    cmpl %edx, %r8d          # while j < N (j - N < 0)
    jge doneWithCells

# Copy the element A points to to C (in same cell location)
    movb (%rdi), %r9b        # temp = element A points to
    movb %r9b, (%rsi)        # cell location C points to = temp

# Update A and C so they point to their next element/cell location
    incq %rdi
    incq %rsi

    incl %r8d                # j++ (column index in r8d)
    jmp colLoop              # go to next cell

# Go to next row
doneWithCells:
    incl %ecx                # i++ (row index in ecx)
    jmp rowLoop              # go to next row

doneWithRows:                # bye! bye!
    ret

#####

# # ***** Version 1 - Original version *****
#     .globl     copy
# copy:
# # A in rdi, C in rsi, N in edx
#     xorl %eax, %eax        # set eax to 0
# # since this function is a leaf function, no need to save caller-
# saved registers rcx and r8
#     xorl %ecx, %ecx        # row number i is in ecx -> i =
0

# # For each row
# rowLoop:
#     movl $0, %r8d          # column number j in r8d -> j = 0
#     cmpl %edx, %ecx        # loop as long as i - N < 0

```

```

#     jge doneWithRows

# # For each cell of this row
# colLoop:
#     cmpl %edx, %r8d           # loop as long as j - N < 0
#     jge doneWithCells

# # Compute the address of current cell that is copied from A to C
# # since this function is a leaf function, no need to save caller-
# saved registers r10 and r11
#     movl %edx, %r10d         # r10d = N
#     imull %ecx, %r10d        # r10d = i*N
#     addl %r8d, %r10d         # j + i*N
#     imull $1, %r10d          # r10 = L * (j + i*N) -> L is char
#                               (1Byte)
#     movq %r10, %r11          # r11 = L * (j + i*N)
#     addq %rdi, %r10           # r10 = A + L * (j + i*N)
#     addq %rsi, %r11           # r11 = C + L * (j + i*N)

# # Copy A[L * (j + i*N)] to C[L * (j + i*N)]
#     movb (%r10), %r9b         # temp = A[L * (j + i*N)]
#     movb %r9b, (%r11)         # C[L * (j + i*N)] = temp

#     incl %r8d                # column number j++ (in r8d)
#     jmp colLoop              # go to next cell

# # Go to next row
# doneWithCells:
#     incl %ecx                # row number i++ (in ecx)
#     jmp rowLoop              # Play it again, Sam!

# doneWithRows:                # bye! bye!
#     ret

#####
        .globl    transpose
transpose:
# void transpose(void *, int ); transpose(C, N);
# C in rdi, N in esi

# This function is not a "caller" i.e., it does not call functions
# (it is a leaf function),
# hence it does not have the responsibility of saving "caller-
# saved" registers
# such as rax, rdi, rsi, rdx, rcx, r8, r9, r10 and r11.

```

This signifies that it can use these registers without first saving their content.

This function is a "callee" hence it does have the responsibility of saving callee-saved" registers

such as r12 before using it and the responsibility of restoring it before returning to "caller" (main).

```
    pushq %r12
```

Set up registers

```
    xorl %eax, %eax
```

set eax to 0

```
    xorl %ecx, %ecx
```

i = 0 (row index i is in ecx)

```
    movl %esi, %edx
```

save N (into %edx)

```
    decl %edx
```

N-1 now in %edx

For each row

rowLoopT:

```
    cmpl %edx, %ecx
```

while i < N-1 (i - N-1 < 0)

```
    jge doneWithRowsT
```

```
    leal 1(%ecx), %r8d
```

j = i+1 (column index j in r8d)

For each cell of this row

colLoopT:

```
    cmpl %esi, %r8d
```

while j < N (j - N < 0)

```
    jge doneWithCellsT
```

Transpose (swap) cell C + L(j + i*N) with C + L(i + j*N)

Compute the address of cell 1 -> C + L(j + i*N)

```
    movl %esi, %r9d
```

temp (r9d) = N

```
    imull %ecx, %r9d
```

temp (r9d) = i*N

```
    addl %r8d, %r9d
```

temp (r9d) = j + i*N

```
#    imull $1, %r9d
```

temp (r9d) = L(j + i*N) - no need to

do this since L=1 (char=1B)

```
    addq %rdi, %r9
```

temp (r9) = C + L(j + i*N)

Save cell 1 to temp (r10b)

```
    movb (%r9), %r10b
```

Compute the address of cell 2 (the transpose) -> C + L(i + j*N)

```
    movl %esi, %r11d
```

temp (r11d) = N

```
    imull %r8d, %r11d
```

temp (r11d) = j*N

```
    addl %ecx, %r11d
```

temp (r11d) = i + j*N

```
#    imull $1, %r11d
```

temp (r11d) = L(i + j*N) - no need

to do this since L=1 -> char = 1B

```

        addq %rdi, %r11                # temp (r11) = C + L(i + j*N)

# Save cell 2 to temp (r12b)
        movb (%r11), %r12b

# Copy temp (r10b) to cell 2 (the transpose) -> C + L(i + j*N)
        movb %r10b, (%r11)

# Copy temp (r12b) to cell 1 -> C + L(j + i*N)
        movb %r12b, (%r9)

        incl %r8d                      # j++ (column index in r8d)
        jmp colLoopT                  # go to next cell

# Go to next row
doneWithCellsT:
        incl %ecx                      # i++ (row index in ecx)
        jmp rowLoopT                  # go to next row

doneWithRowsT:
# Stack clean up
        popq %r12                     # restore "callee-saved" register
before returning to "caller" (main)
        ret

#####
        .globl    reverseColumns
# ***** Version 2 *****
reverseColumns:
# void reverseColumns(void *, int n); reverseColumns(C, N);
# C in rdi, N in esi

# This function is not a "caller" i.e., it does not call functions
# (it is a leaf function),
# hence it does not have the responsibility of saving "caller-saved"
# registers
# such as rax, rdi, rsi, rdx, rcx, r8, r9, r10 and r11.
# This signifies that it can use these registers without first
# saving their content.

# This function is a "callee" hence it does have the responsibility
# of saving callee-saved" registers
# such as r12 and r13 before using it and the responsibility of
# restoring it before returning to "caller" (main).
        pushq %r12

```

```

    pushq %r13

# Set up registers
    xorl %eax, %eax                # set eax to 0

    movl %esi, %edx                # save N into %edx
    decl %edx                      # N-1 now in %edx
    movl %esi, %r13d              # save N into %r13d
    shrl $1, %r13d                # N/2 now in %r13d

    xorl %r8d, %r8d                # j = 0 (column index j in r8d)

# For each column
nextColumnPairLoop:
    cmpl %r13d, %r8d              # while j < floor(N/2) (i - floor(N/2)
< 0)
    jge doneReversing

# Set up loop variable i
    xorl %ecx, %ecx                # i = 0 (row index i is in ecx)

# Reverse the cells of this row
# Compute the address of cell 1 -> C + j
    movl %r8d, %r9d               # temp (r9d) = j
    addq %rdi, %r9                # temp (r9) = C + L(j + i*N), but L=1
and i=0

# Compute the address of cell 2 (the reverse) -> C + (N-1-j)
    movl %edx, %r11d              # temp (r11d) = N-1
    subl %r8d, %r11d              # temp (r11d) = N-1-j
    addq %rdi, %r11               # temp (r11) = C + L((i*N) + (N-
1-j)), but L=1 and i=0

# Swap each cell in these 2 columns
nextElementPairLoop:
    cmpl %esi, %ecx                # while i < N (i - N < 0)
    jge doneWithColumnPair

# Save cell 1 to temp (r10b)
    movb (%r9), %r10b
# Save cell 2 to temp (r12b)
    movb (%r11), %r12b
# Copy temp (r10b) to C+ L((i*N) + (N-1-j))
    movb %r10b, (%r11)
# Copy temp (r12b) to C + L(j + i*N)
    movb %r12b, (%r9)

```

```

# Go to next element in both columns ...
    leaq (%rsi, %r9), %r9    # ... by adding N to address of current
    element of column
    leaq (%rsi, %r11), %r11 # ... by adding N to address of current
    element of column

    incl %ecx                # i++
    jmp nextElementPairLoop  # go to next element

# Go to next row
doneWithColumnPair:
    incl %r8d                # j++
    jmp nextColumnPairLoop   # go to next row

doneReversing:
# Stack clean up - reverse order
    popq %r13                # restore "callee-saved" register
before returning to "caller" (main)
    popq %r12
    ret

# #####
#     .globl     reverseColumns
# # ***** Version 1 *****
# reverseColumns:
# # void reverseColumns(void *, int n); reverseColumns(C, N);
# # C in rdi, N in esi

# # This function is not a "caller" i.e., it does not call functions
# # (it is a leaf function),
# # hence it does not have the responsibility of saving "caller-
# # saved" registers
# # such as rax, rdi, rsi, rdx, rcx, r8, r9, r10 and r11.
# # This signifies that it can use these registers without first
# # saving their content.

# # This function is a "callee" hence it does have the
# # responsibility of saving callee-saved" registers
# # such as r12 and r13 before using it and the responsibility of
# # restoring it before returning to "caller" (main).
#     pushq %r12
#     pushq %r13

# # Set up registers
#     xorl %eax, %eax                # set eax to 0

```

```

#    xorl %ecx, %ecx                # i = 0 (row index i is in ecx)

#    movl %esi, %edx                # save N into %edx
#    decl %edx                     # N-1 now in %edx
#    movl %esi, %r13d              # save N into %r13d
#    shrl $1, %r13d                # N/2 now in %r13d

# # For each row
# rowLoopR:
#    cmpl %esi, %ecx                # while i < N (i - N < 0)
#    jge doneWithRowsR

#    xorl %r8d, %r8d                # j = 0 (column index j in r8d)

# # For each cell of this row
# colLoopR:
#    cmpl %r13d, %r8d              # while j < floor(N/2) (i - floor(N/2)
#    < 0)
#    jge doneWithCellsR

# # Reverse the cells of this row
# # Compute the address of cell 1 -> C + L(j + i*N)
#    movl %esi, %r9d               # temp (r9d) = N
#    imull %ecx, %r9d              # temp (r9d) = i*N
#    addl %r8d, %r9d               # temp (r9d) = j + i*N
# #    imull $1, %r9d              # temp (r9d) = L(j + i*N) - no need to
# do this since L=1 (char=1B)
#    addq %rdi, %r9                # temp (r9) = C + L(j + i*N)

# # Save cell 1 to temp (r10b)
#    movb (%r9), %r10b

# # Compute the address of cell 2 (the reverse) -> C + L((i*N) + (N-
# 1-j))
#    movl %esi, %r11d              # temp (r11d) = N
#    imull %ecx, %r11d             # temp (r11d) = i*N
#    movl %edx, %r12d              # temp (r12d) = N-1
#    subl %r8d, %r12d              # temp (r12d) = N-1-j
#    addl %r12d, %r11d             # temp (r11d) = (i*N) + (N-1-j)
# #    imull $1, %r11d             # temp (r11d) = L(i + j*N) - no need to
# do this since L=1 -> char = 1B
#    addq %rdi, %r11               # temp (r11) = C + L((i*N) + (N-
# 1-j))

# # Save cell 2 to temp (r12b)
#    movb (%r11), %r12b

```



```
# # Copy temp (r10b) to C+ L((i*N) + (N-1-j))
#     movb %r10b, (%r11)

# # Copy temp (r12b) to C + L(j + i*N)
#     movb %r12b, (%r9)

#     incl %r8d                # j++ (column index in r8d)
#     jmp colLoopR            # go to next cell

# # Go to next row
# doneWithCellsR:
#     incl %ecx                # i++ (row index in ecx)
#     jmp rowLoopR            # go to next row

# doneWithRowsR:
# # Stack clean up - reverse order
#     popq %r13                # restore "callee-saved" register
before returning to "caller" (main)
#     popq %r12
#     ret
```
